

Properties

The `BankAccount` class that we used in Chapter 15 marked its `_balance` attribute as private by placing an underscore at the start of its name. But remember that designating an attribute as private is only a convention: all attributes in Python are technically public, meaning they're accessible to code outside the class. There's nothing to prevent code from intentionally or maliciously changing the `_balance` attribute to an invalid value.

But you can prevent *accidental* invalid changes to these private attributes with properties. In Python, *properties* are attributes that have specially assigned *getter*, *setter*, and *deleter* methods that can regulate how the attribute is read, changed, and deleted. For example, if the attribute is only supposed to have integer values, setting it to the string `'42'` will likely cause bugs. A property would call the setter method to run code that fixes, or at least provides early detection of, setting an invalid value. If you've thought, "I wish I could run some code each time this attribute was accessed, modified with an assignment statement, or deleted with a `del` statement," then you want to use properties.

Turning an Attribute into a Property

First, let's create a simple class that has a regular attribute instead of a property. Open a new file editor window and enter the following code, saving it as *regularAttributeExample.py*:

```
class ClassWithRegularAttributes:
    def __init__(self, someParameter):
        self.someAttribute = someParameter

obj = ClassWithRegularAttributes('some initial value')
print(obj.someAttribute) # Prints 'some initial value'
obj.someAttribute = 'changed value'
print(obj.someAttribute) # Prints 'changed value'
del obj.someAttribute # Deletes the someAttribute attribute.
```

This `ClassWithRegularAttributes` class has a regular attribute named `someAttribute`. The `__init__()` method sets `someAttribute` to `'some initial value'`,